Department of Computer Science, University of Karachi

PDFSage: An NLP-Based Intelligent PDF Assistant

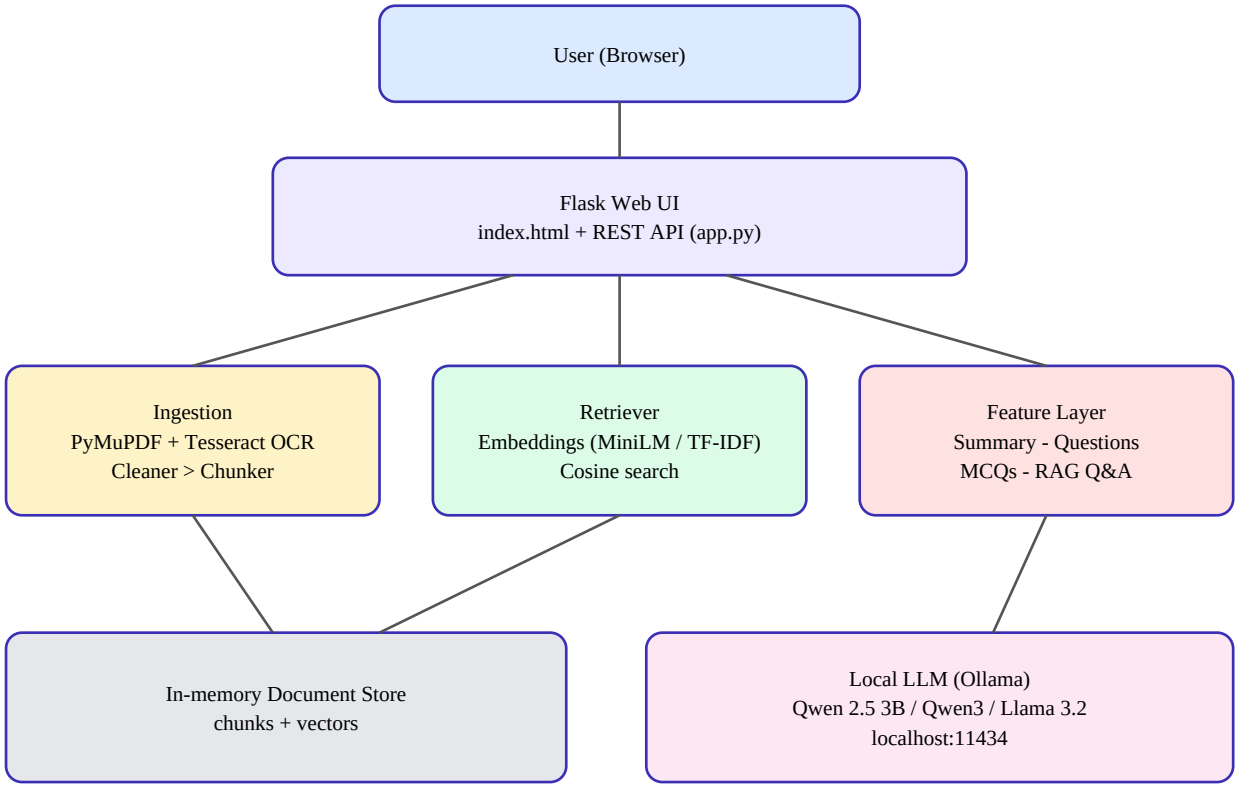
System Design & Test Report

Course	CSSE 606: Topic of Current Interest III
Submitted to	Dr. M. Saeed
Group members	Alishna Azhar (B2211016013) Doshab Hussain (B2211016021) Huzaifa Arain (B2211016028) Hassaan Mushtaq (B2211016049)

1. Overview

PDFSage turns a PDF (ideally 20-25 pages) into a summary, study questions, MCQs and a chat interface grounded in the document. Everything runs locally: a small LLM (about 3B parameters, e.g. Qwen 2.5 3B) served by Ollama, PyMuPDF for parsing and OCR, and a Flask web UI. No external API or key is used.

2. Architecture



3. Components

Module	File	Responsibility	NLP concept
PDF Loader	backend/pdf_loader.py	Text extraction with PyMuPDF; OCR for scanned pages; cleaning	Preprocessing, OCR
Chunker	backend/chunker.py	Sentence-aware ~180-word chunks, 1-sentence overlap	Segmentation
Retriever	backend/retriever.py	Embeds chunks, cosine search, representative/spread sampling	Semantic search
LLM Client	backend/llm.py	Talks to local Ollama; JSON mode; strips reasoning tags	LLM, text generation
Features	backend/features.py	Summary, questions, MCQs, grounded answers; JSON repair + retry	Summarization, QG, RAG
Web App	app.py, templates/index.html	Flask REST API and dark responsive UI	-

4. Data Flow

Ingest: upload > PyMuPDF text; pages with under 40 characters are OCR'd (Tesseract) > clean > chunk > embed > in-memory store.
Summary: chunks closest to the document centroid (extractive selection) > LLM writes bullets (abstractive).
Questions / MCQs: chunks sampled evenly across the document > LLM in JSON mode > validated (4 options, valid answer index), retried once on failure.
Chat (RAG): query embedding > top-3 chunks > LLM answers using only that context; sources are shown in the UI.

5. REST API

Endpoint	Method	Purpose
/api/status	GET	Ollama online, installed models, loaded document info
/api/model	POST	Switch local model
/api/upload	POST	PDF > extract/OCR > chunk > embed
/api/summary	POST	Bullet-point summary
/api/questions	POST	Study questions (n)
/api/mcqs	POST	MCQs: 4 options + answer index (n)
/api/chat	POST	RAG answer + source passages

6. Design Decisions

- Small local model: context kept to 4096 tokens, so prompts use 3-6 chunks of ~180 words.
- JSON mode plus validation makes a 3B model reliable for MCQs.
- Embeddings: MiniLM (sentence-transformers) with automatic TF-IDF fallback.
- Single-user in-memory state is enough for a local desktop tool.

7. Testing Against the Design

Test	Verifies (design element)
test_text_extraction	PDF Loader: text layer, page count
test_ocr_on_scanned_pdf	PDF Loader: OCR of an image-only PDF (PyMuPDF + Tesseract)
test_chunking_and_retrieval	Chunker + Retriever: relevant chunk ranked first
test_features	Feature layer: summary, questions, MCQs, RAG answer
test_bad_json_is_handled	Robustness: malformed LLM JSON repaired or safely empty
test_ollama_client_against_fake_server	LLM client: payload (JSON mode, think off), tag stripping, model list
test_ollama_offline_message	LLM client: clear error when Ollama is not running
test_flask_end_to_end	REST API: upload validation, all features, error codes

Result: 8 passed in 3.25s

The Ollama protocol is tested against a mock server; output quality of the real model depends on the model installed on your PC.

8. Limitations and Future Work

English-only OCR by default; one document at a time; long documents are summarized from representative chunks (map-reduce would improve coverage); add FAISS and multi-PDF support.